



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

► **Curso: Fundamentos de Programación**

► **Docente: Américo Chulluncuy Reynoso**

2025-3

Sesión 13:

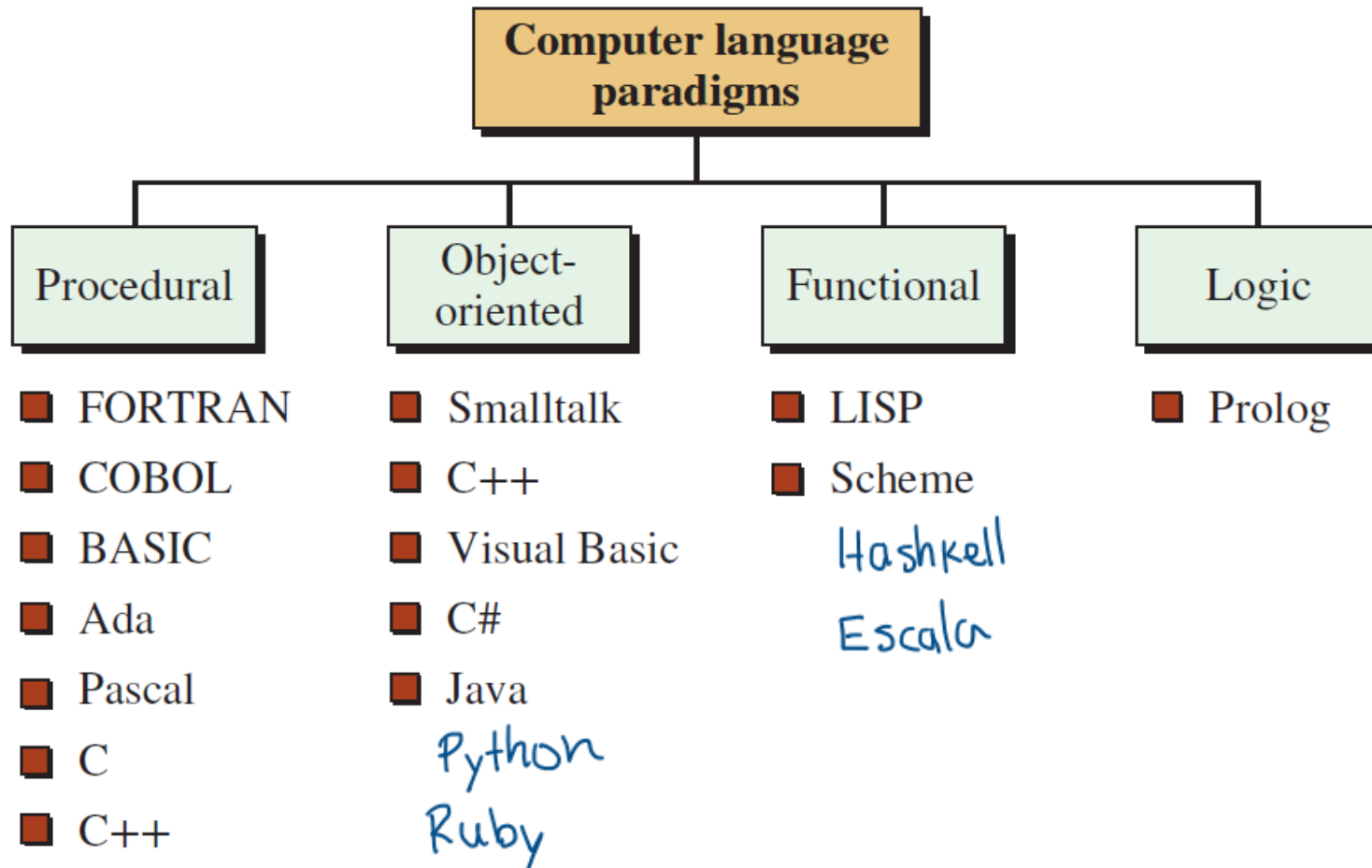
Clases

Contenido

1. Introducción
2. Clases y objetos
3. Constructores y destructores
4. Puntero this:
5. Uso del puntero this en un constructor
6. Tarea

1. Introducción

- Los lenguajes informáticos se pueden clasificar según el enfoque que utilizan para resolver un problema.
- Un **paradigma** es un modelo para describir cómo un programa maneja los datos.

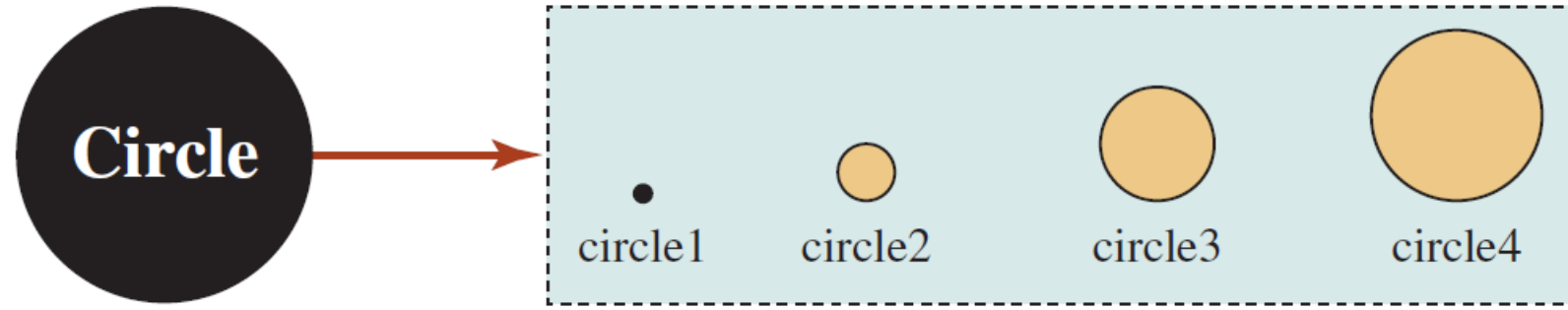


Paradigmas de lenguaje

- **Programación procedural:** El programa se estructura en torno a procedimientos o funciones que manipulan datos.
- **Programación Orientada a Objetos (POO):** Gira en torno al concepto de objetos, que son instancias de clases. Los objetos interactúan entre sí mediante métodos, este enfoque facilita la reutilización de código, modularidad y el mantenimiento.
- **Programación funcional:** trata la computación como la evaluación de funciones matemáticas y evita cambios de estado y datos mutables.
- **Programación lógica:** Utiliza un conjunto de hechos y un conjunto de reglas para responder consultas. Se basa en la lógica formal.

2. Clases y Objetos

- Un **tipo** es una **abstracción**; una **instancia** de ese tipo es una **entidad concreta**. Ejemplo:



- La relación entre un tipo y sus instancias es una **relación de uno a muchos**.
- Cualquier instancia (real) tiene:
 - Atributos**: es cualquier característica que nos interese
 - Comportamientos**: operación que la instancia puede realizar sobre sí misma.

Atributos y comportamientos

Ejemplo: Atributos

- Instancia de Circle: radio, perímetro, área.
- Instancia de Empleado: nombre, dirección, salario.
- Instancia de Alumno: ?

Ejemplo: Comportamientos

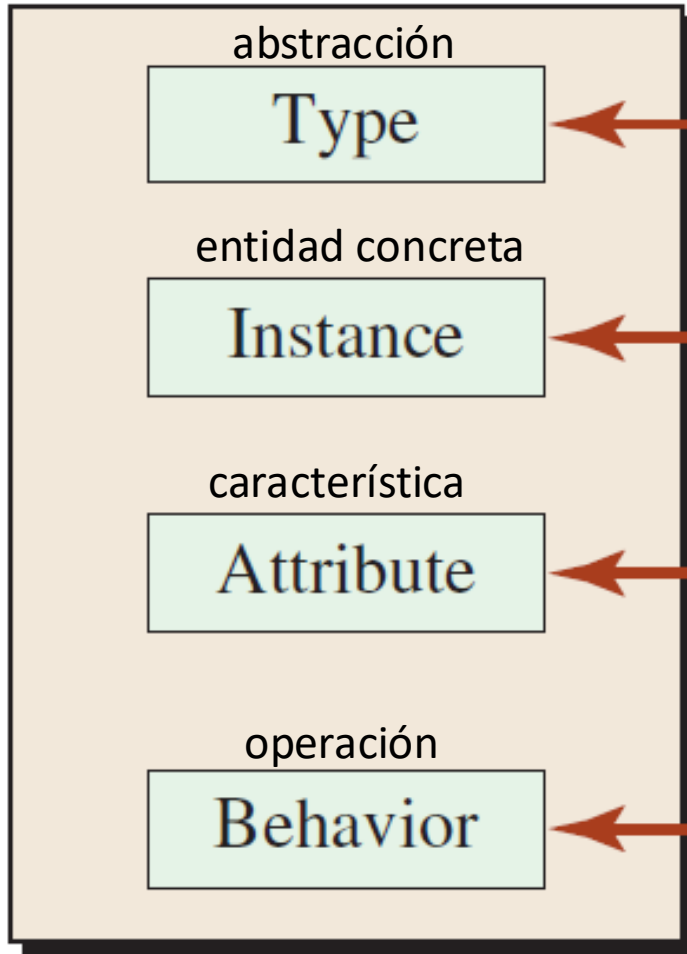
- Instancia de Circle: puede dar su radio, su perímetro y su área.
- Instancia de Empleado: esta puede dar su nombre, dirección y salario.
- Instancia de Alumno: ?

Clases y objetos en programas

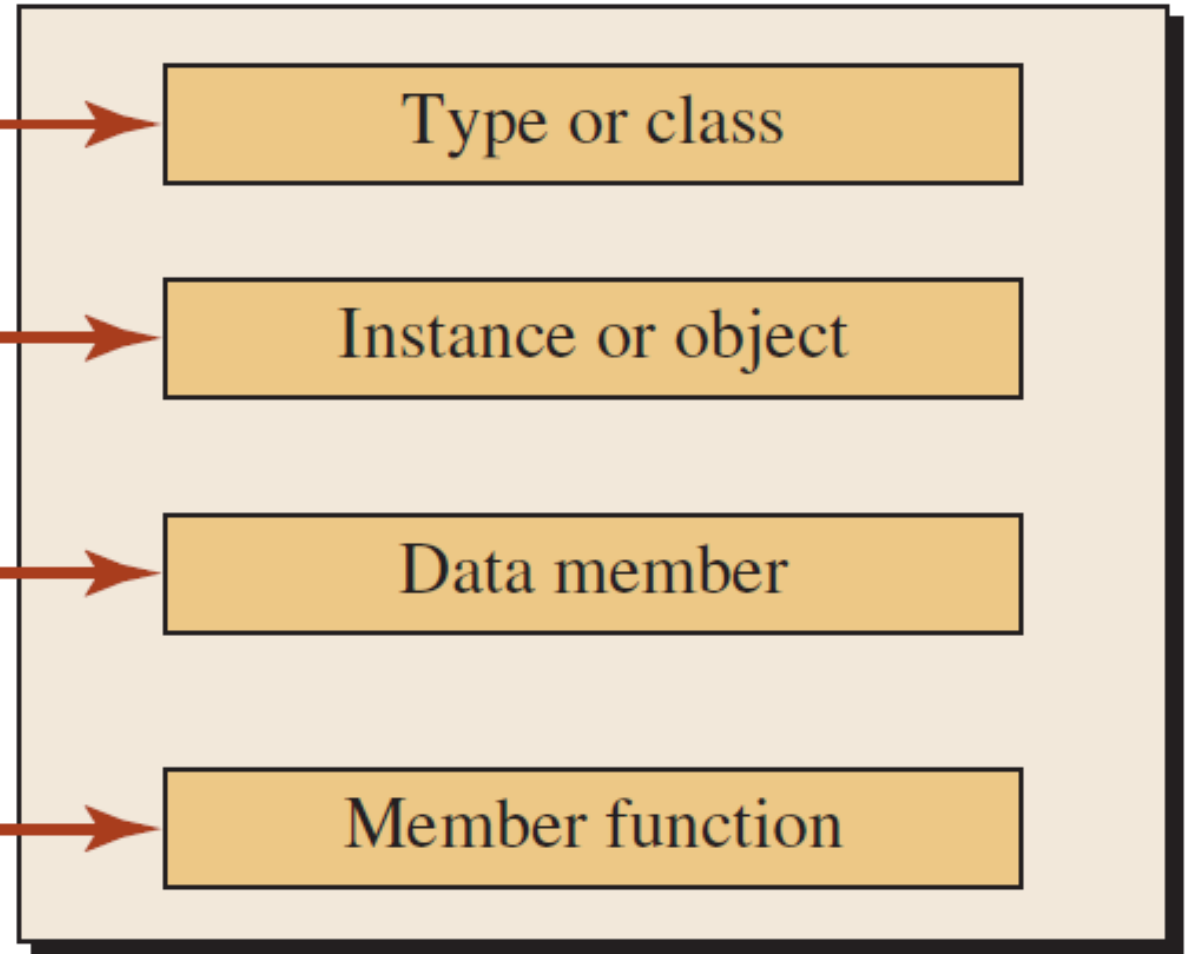
- En POO, creamos un tipo (definido por el usuario) utilizando **class**. Una instancia de una clase se denomina **objeto**.
- En POO, los atributos y comportamientos de un objeto son representados como miembros de datos y funciones miembros.
 - ✓ Un **miembro de dato** de un objeto es una variable cuyo valor representa un atributo. Por ejemplo, en el caso del círculo, `double radio;`
 - ✓ Una **función miembro** (método) simula uno de los comportamientos de un objeto. Por ejemplo, podemos escribir una función que permita a un círculo dar su radio.

Comparación de términos

Realidad



POO



Clases

- Una clase es una colección de un número fijo de componentes llamados **miembros** de la clase.
- Sintaxis para **definir** una clase

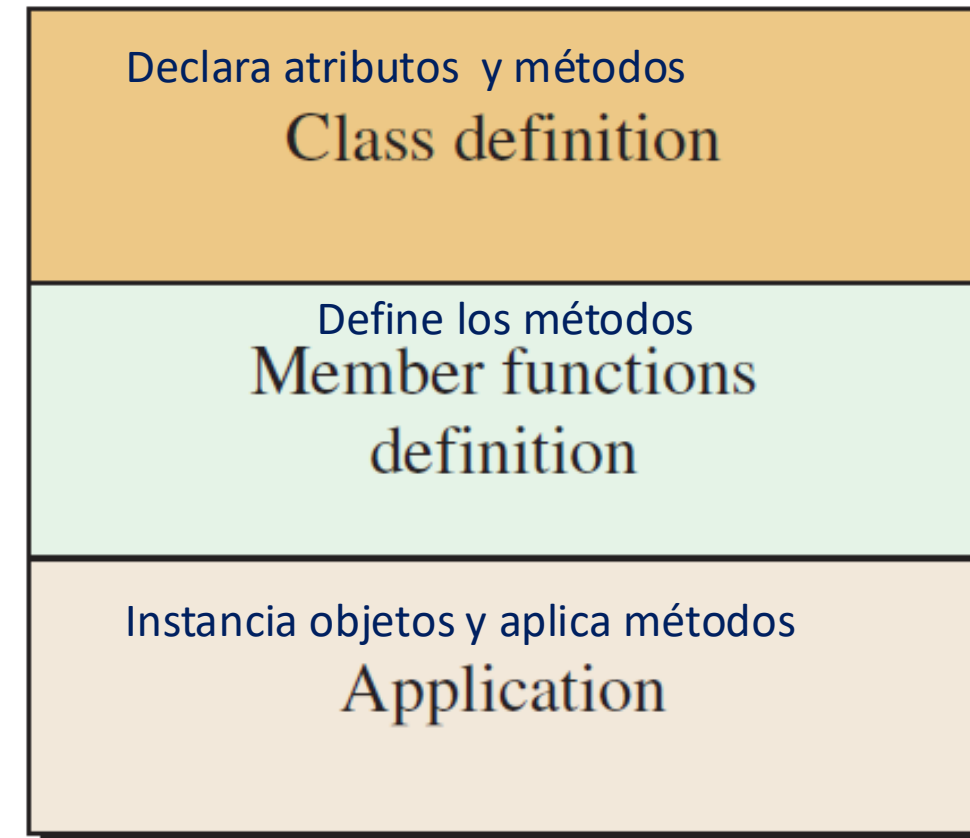
```
class IdentificadorClase{  
    ListaMiembrosClase  
};
```

- ListaMiembrosClase consta de declaraciones de variables y/o funciones, es decir, un miembro de una clase puede ser una variable (para almacenar datos) o una función (para manipular datos).

Programas en el paradigma orientado a objetos

- A diferencia de la programación procedimental, en POO necesitamos **tres secciones**:
 - ✓ **Sección 1: Definición de clase,**
 - ✓ **Sección 2: Definición de función miembro**
 - ✓ **Sección 3: Aplicación**
- En algunos casos cada sección es diseñada y almacenada por diferentes entidades.

Por ejemplo en las clases string y fstream, las 2 primeras secciones son creadas y almacenadas en la biblioteca estándar de C++.



Sección 1: definición de clase

- Consta de tres partes: encabezado, cuerpo y punto y coma. Ejemplo:

```
class Circle {  
private: //modificador de acceso (de grupo)  
    double radius; //declaración de miembros de datos o variables (Atributos)  
           //variables integradas u otros tipos de clases previamente definidas  
public:  
    double getRadius () const; //declaración de funciones miembro (Métodos)  
    double getArea () const; //const, significa que la función no puede modificar los  
    double getPerimeter () const; //miembros de datos de una variable de tipo Circle  
    void setRadius (double value);  
};
```

- El **modificador de acceso** determina cómo se puede acceder a una clase. Por defecto es **private** (no se puede acceder a los datos y funciones miembros para recuperarlos o modificarlos)

Modificadores de acceso

- Para evitar esta limitación, C++ define tres modificadores de acceso:

Modifier	Access from same class	Access from subclass	Access from anywhere
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes

- Los **miembros de datos** normalmente se configuran como **private**. Se debe acceder a ellos a través de las funciones miembros.
- Las **funciones miembros** normalmente deben establecerse como **public**.

Sección 2: Definición de funciones miembro

- Cada función miembro necesita una definición usual, **con dos diferencias:** (i) const y (ii) el nombre de la función que debe calificarse con el nombre de la clase seguido de un símbolo de alcance de clase ::

```
double Circle :: getRadius () const { //la función no puede modificar el objeto
    return radius;
}
double Circle :: getArea () const {
    const double PI = 3.14;
    return (PI * radius * radius);
}
double Circle :: getPerimeter () const {
    const double PI = 3.14;
    return (2 * PI * radius);
}
void Circle :: setRadius (double value) {
    radius = value;
}
```

Funciones en línea

- El tiempo de ejecución asociado a una llamada de función puede ser mayor que ejecutar directamente el código contenido en la función. Para mejorar el rendimiento, se puede declarar una función como función en línea (inline), lo que **sugiere** al **compilador reemplazar la llamada por el cuerpo de la función en tiempo de compilación**.
- Una **función en línea implícita** ocurre cuando se define directamente dentro de la declaración de la clase. Aunque es válida no se recomienda porque:
 - (i) Dificulta la legibilidad de código.
 - (ii) Puede violar el principio de encapsulamiento (***?!)**
- Una **función en línea explícita** se define utilizando la palabra clave `inline` antes de su definición, generalmente fuera de la clase. Esto permite mantener la separación entre interfaz e implementación, preservando así una mejor organización del código.

Sección 3: Aplicación

- Utiliza la definición de clase y la definición de función miembro. **Crea instancias de objetos de la clase y aplica las funciones miembros en esos objetos**

```
Circle circle1;
```

```
circle1.setRadius (10.0);
```

```
cout << "Radio: " << circle1.getRadius() << endl;
```

```
cout << "Area: " << circle1.getArea() << endl;
```

```
cout << "Perimetro: " << circle1.getPerimeter() << endl;
```

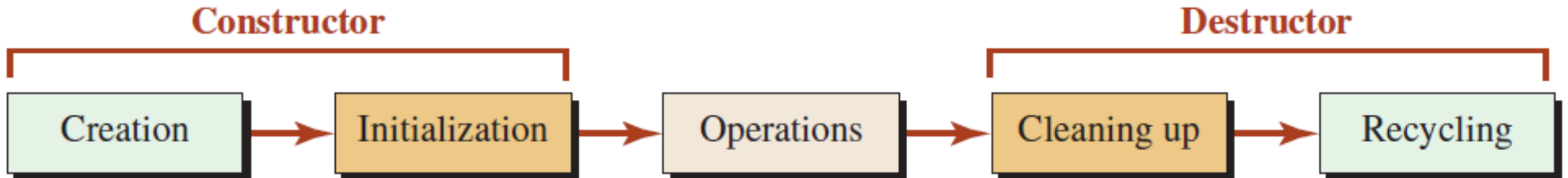
- **Ejemplo1**

3. Constructores y destructores

- Para que un objeto realice operaciones sobre sí mismo, primero debe crearse. La **creación ocurre al invocar el constructor**. Durante la construcción:
 - ✓ Primero se ejecuta la lista de inicialización (antes del cuerpo del constructor).
 - ✓ Luego se ejecuta el cuerpo del constructor para realizar otras tareas opcionales.
- Cuando un objeto ya no es necesario, sus recursos deben de liberarse correctamente. Para ello existe otra función especial llamada destructor. El destructor se ejecuta automáticamente **cuando el objeto sale de su alcance**.

Constructores y destructores

- Por lo tanto, un **objeto pasa por cinco pasos**. Es **creado e inicializado** por una función miembro especial llamada constructor. **Aplica algunas operaciones** solicitadas por la aplicación sobre sí misma. Es **limpiado y reciclado** por otra función miembro especial llamada destructor.



Constructor

- Es una función miembro que **crea un objeto cuando se llama e inicializa los datos miembros de un objeto cuando se ejecuta.**
- Un constructor **tiene dos características:** (1) **no tiene valor de retorno** y su (2) **nombre es el mismo que el nombre de la clase.**
- Podemos tener **tres tipos de constructores en una clase:** constructores de parámetros, constructores predeterminados y constructores de copia.
- **Declaración:**

```
class Circle{  
    public:  
        Circle (double radius); //Constructor de parámetros  
        Circle (); //Constructor predeterminado  
        Circle (const Circle& circle); //Constructor de copia  
}
```

Tipos de constructores

- Un **constructor de parámetros** inicializa los miembros de datos de cada instancia con valores específicos. Podemos tener varios constructores de parámetros, cada uno con un valor diferente (sobrecarga).
- El **constructor predeterminado** no tiene parámetros. Útil para crear objetos donde cada miembro de datos es establecido en algunos valores literales o **valores predeterminados**. No lo podemos sobrecargar.
- Un **constructor de copia**, copia los valores de los miembros de datos del objeto dado al nuevo objeto recién creado. **Tiene solo un parámetro por referencia** (al objeto fuente). No lo podemos sobrecargar porque la lista de parámetros es fija. No podemos cambiar el objeto fuente

Constructor: Definición

```
// Definición de constructor de parametros
Circle :: Circle (double rds)
: radius (rds) // lista de inicialización de
               //miembros antes del cuerpo
{
// ...
}
// Definición de constructor predeterminado
Circle :: Circle ()
: radius (1.0) // Lista de inicialización.
{
// ...
}
// Definición de constructor copia
Circle :: Circle (const Circle& cr)
: radius (cr.radius) // lista de inicialización
{
// ...
}
```

- Note que un constructor **puede tener una lista de inicialización después del encabezado para inicializar los miembros de datos.**
- Si necesitamos inicializar más de un miembro de dato, **la inicialización de cada miembro de datos debe estar separada por una coma de otros miembros de datos.**
- El nombre de cada parámetro lo determina el programador, pero el del miembro de datos debe ser el mismo
- **Podemos usar el cuerpo del constructor para inicializar miembros de datos complejos, para validar un parámetro, para abrir archivos si es necesario o incluso imprimir un mensaje para verificar que se llamó al constructor.**

Destructor

- Es una función miembro que **limpia y destruye un objeto**. Tiene **dos características especiales**.
 1. Tiene el mismo nombre de la clase precedido por ~
 2. No puede tener un valor de retorno
- El sistema llamará y ejecutará automáticamente un destructor cuando el objeto instanciado de la clase (**en el stack**) salga del alcance.
- La limpieza es importante si lo construido ha llamado recursos como archivos. Una vez finalizado el programa, la memoria asignada se recicla.
- Un destructor no puede aceptar argumentos, lo que significa que no puede sobrecargarse.

Destructor: declaración y definición

- Declaración:

```
class Circle {  
    public:  
        ~Circle ();  
}
```

- Definición:

```
Circle :: ~Circle () {  
    // ...  
}
```

- Llamada

**Parameter
constructor**

Circle circle1 (5.1);

**Default
constructor**

Circle circle2;

**Copy
constructor**

Circle circle3 (aCircle);

Destructor

Called by system

Ejemplo: uso de constructores y destructor

Creación de variables vs objetos de clases

La siguiente tabla compara la creación de variables de tipos integrados con la creación de objetos de tipos clase.

Member	Class type	Built-in type
Parameter constructor	Circle circle1 (10.0);	double x1 = 10.0;
Default constructor	Circle circle2;	double x2;
Copy constructor	Circle circle3 (circle1);	None
Destructor	No call	No call

Funciones miembro requeridas por una clase

- **G1. Constructor de parámetros y el constructor predeterminado.** Debemos tener al menos uno de estos constructores. Si proporcionamos cualquiera de ellos, el sistema no nos proporciona ninguno. Si no proporcionamos ninguno de ellos, el sistema proporciona un constructor predeterminado, que inicializa cada miembro con lo que encuentran en el sistema (ver [Ejemplo 1](#)).
- **G2. Constructor de copias.** Una clase debe tener **un único constructor de copia**, pero si no proporcionamos uno, el sistema nos proporciona uno. La mayoría de las veces **es mejor crear nuestro propio constructor de copias**.
- **G3. Destructor.** Una clase debe tener **un solo destructor**, pero si no proporcionamos uno, el sistema nos proporciona uno. **Es mejor crear nuestro propio destructor**.

Ejemplo 2

En el [Ejemplo 2](#), volvemos a considerar el Ejemplo 1 de la clase Circle ahora con los constructores y el destructor proporcionados. Incluimos un mensaje en el cuerpo de cada constructor y destructor para mostrar cuándo se llaman

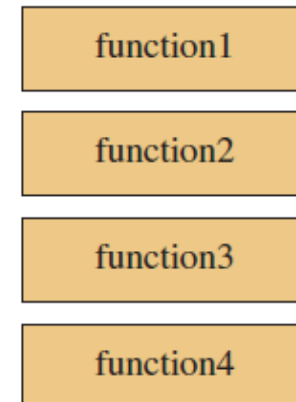
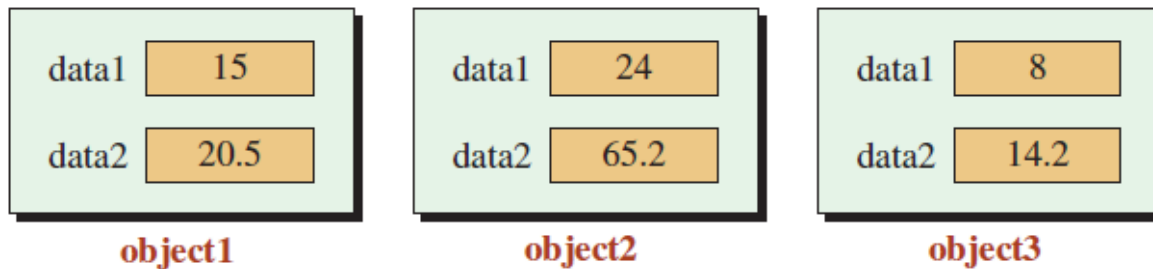
```
class Circle {
    private:
        double radius; //declaración de miembros de datos
    (variables)
    public:
        Circle(double radius); // Constructor de parámetro
        Circle(); //Constructor predeterminado
        ~Circle(); //Destructor
        Circle(const Circle& circle); // constructor copia
        void setRadius (double value); //Mutador
        double getRadius () const; // Accesor
        double getArea () const; //Accesor
        double getPerimeter () const; // Accesor
};
```

Ejemplo 3: Generando números aleatorios

- En este caso solo necesitamos definir el constructor de parámetros (el sistema no crea un constructor predeterminado) porque los constructores predeterminados y de copia no tienen sentido.
- delete evita que el sistema proporcione un constructor de copia (C++11)
- Solo dos de los tres miembros de datos (low y high) se inicializan en la lista de inicialización del constructor. El tercer miembro de datos (valor) se calcula en el cuerpo del constructor.
- La única función miembro accesor es la función `print`, que imprime el valor del número aleatorio creado. No tenemos funciones miembros mutadoras porque no queremos cambiar el número aleatorio creado.

Miembros de un objeto

- **Miembros de datos:** define los atributos del objeto. Cada objeto debe **encapsular** (se asignan regiones de memoria separadas para cada objeto) el conjunto de miembros de datos definidos en la clase. Otros objetos no pueden acceder a ellos. Normalmente es privado
- **Funciones miembros:** define uno de los comportamientos que se pueden aplicar en los miembros de datos de un objeto. **Solo hay una copia de cada función miembro** del objeto en la memoria y debe compartirse por todas las instancias. Normalmente es público.

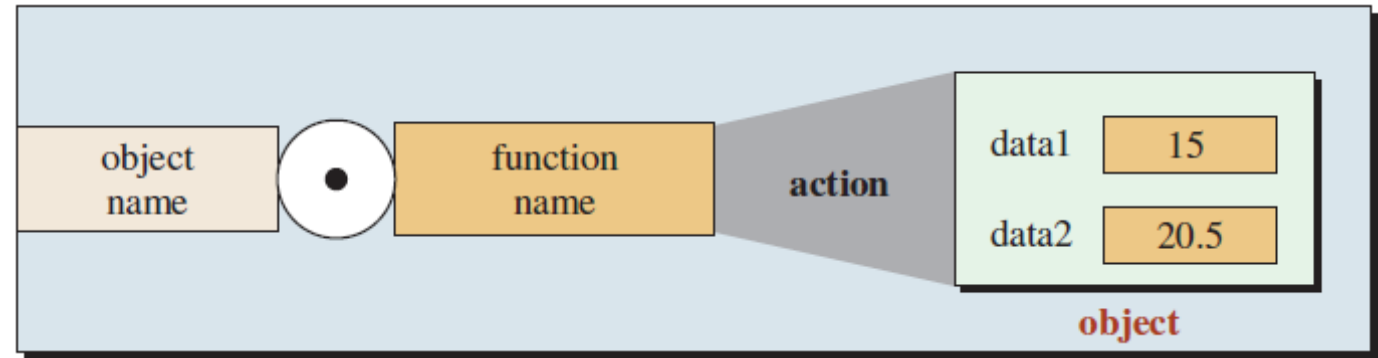


(*?! ok)

Selectores de funciones miembros

- Una aplicación puede llamar a una función miembro de instancia para operar en ella. En POO esta llamada debe realizarse a través de la instancia (objeto)
- La aplicación primero debe construir una instancia y luego dejar que la instancia llame a la función miembro. El lenguaje C++ define dos operadores, llamados operadores selectores de miembros, para este propósito.

Operator	Expression
.	object.member
->	pointer -> member



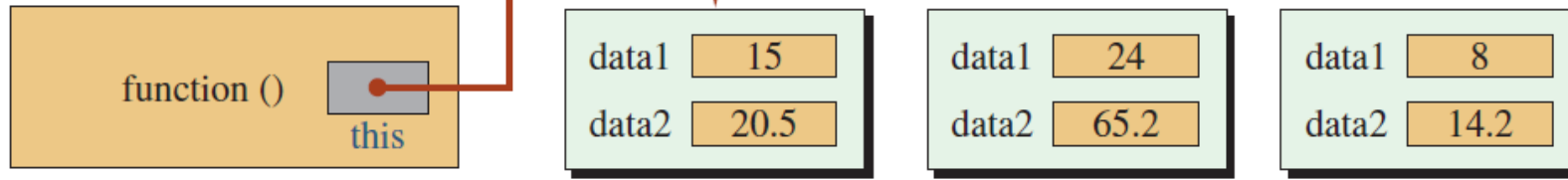
- Ver Ejemplos 2 y 3

Puntero this

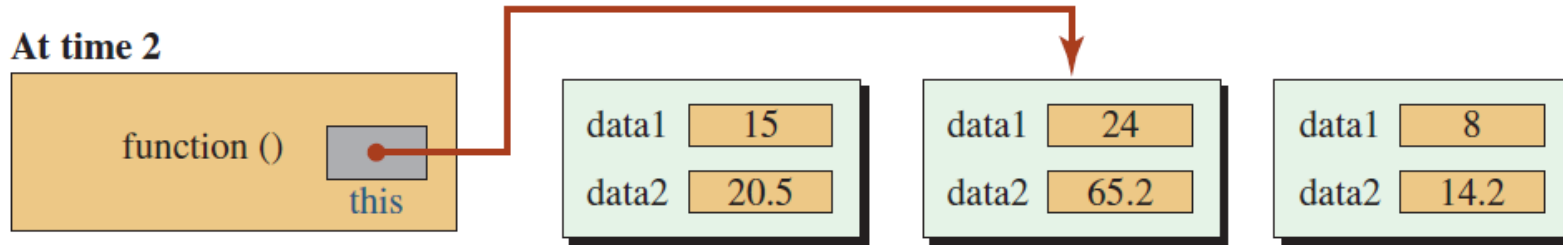
- Dado que solo hay una copia de una función miembro, para bloquear una función cuando se está utilizando y desbloquearla cuando la función finaliza, C++ agrega un puntero a cada función miembro.
- Mientras usamos el selector de miembros de punto, **el compilador lo cambia a un selector de miembros de puntero**, en el que **cada función miembro tiene un puntero oculto** llamado puntero **this**.
- La función es empleada por el objeto al que apunta este puntero en ese momento (el código de función se aplica a los miembros de datos del objeto al que apunta el puntero this)

Puntero this

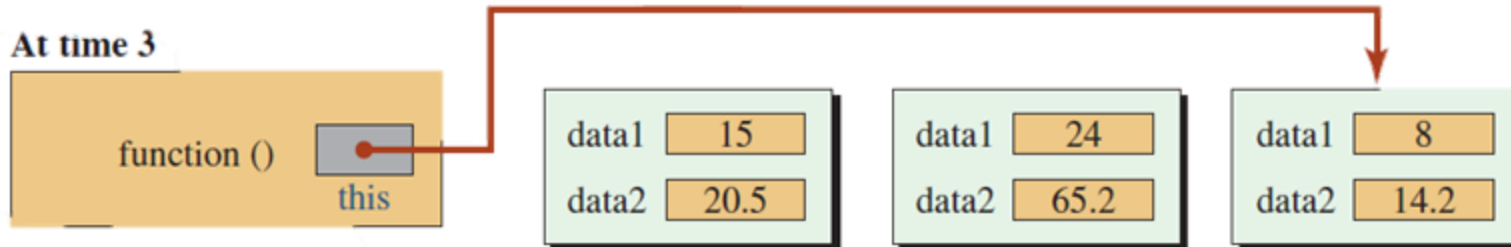
At time 1



At time 2



At time 3



Bloqueo y desbloqueo de una función para un objeto particular

Puntero this: uso implícito

- ¿Cómo obtiene una función miembro el puntero this?

El compilador lo agrega como parámetro a la función miembro de la instancia como se muestra a continuación:

```
// Lo que el usuario escribe
```

```
double getRadius () const{  
    return radius;  
}
```

```
circle1.getRadius();
```

```
// Cambio del compilador
```

```
double getRadius (Circle* this)const{  
    return (this -> radius);  
}
```

```
this = &circle1;  
getRadius (this);
```

- **Recordemos:** `this -> radius` es equivalente `(*this).radius`

Puntero this: uso explícito

- Podemos usar el puntero this en nuestro programa para **hacer referencia a un miembro de datos** en lugar de usar el miembro de datos en sí, y podemos usar el nombre del miembro de datos como parámetro.
- De esta forma, no tenemos que utilizar nombres abreviados

```
// sin usar el puntero this
```

```
void Circle :: setRadius (double rds){  
    radius = rds;  
}
```

```
// usando el puntero this
```

```
void Circle :: setRadius (double radius) {  
    this -> radius = radius;  
}
```

Puntero this: uso explícito

- **objeto anfitrión** (host) Es un **objeto sobre el que está operando la función miembro en un momento dado.**
- El puntero this no se puede utilizar en la lista de inicialización de un constructor porque en ese momento el objeto anfitrión no ha sido construido.
- Sin embargo, se puede utilizar en el cuerpo del constructor si es necesario.
- El puntero this es un `unique_ptr` (Smart pointers)

5. Uso del puntero this en un constructor

- this apunta al objeto en el que se está trabajando actualmente. Dentro de sus funciones miembro, se utiliza para hacer referencia a los miembros y métodos del objeto actual. En general, todas las funciones miembros pueden acceder a este puntero, que el compilador crea automáticamente.
- El constructor de una clase es un lugar donde se utiliza con frecuencia el puntero this. Cuando se crea un objeto de la clase, se invoca el constructor.
- Se utiliza para configurar requisitos adicionales e inicializar los miembros de datos del objeto. Es útil para distinguir entre los parámetros del constructor y los miembros de datos del objeto.

Ejemplo

```
#include <iostream>
using namespace std;
class MyClass {
    private:
        int value;
    public:
        MyClass(int value) { //para diferenciar entre parámetro local y variable miembro
            this->value = value; // Inicializamos la variable miembro usando el puntero 'this'
        }
        void printValue() { //para acceder a la variable miembro e imprimir su valor
            cout << "Value: " << this->value << endl;
        }
};

int main() {
    MyClass obj(10);
    obj.printValue();
    return 0;
}
```

Adicional:

Función miembro getters

- Una función miembro de acceso obtiene información del objeto anfitrión(actual) pero no cambia el estado del objeto, es decir, convierte el objeto anfitrión en un objeto de sólo lectura.
- Para garantizar que una función miembro de acceso no cambie el estado del objeto, **debemos agregar el calificador `const`** al final del encabezado de la función (tanto en la declaración como en la definición)

```
//el calificador const hace que el objeto anfitrión sea solo de lectura  
double getRadius () const;  
double getPerimeter () const;  
double getArea() const;
```

- Una función miembro de acceso no tiene que devolver el valor de un miembro de datos; se puede utilizar para generar un valor, siempre que no haya cambios en el estado del objeto. Ejemplo, podemos tener una función de salida que imprima el valor del radio, el perímetro y el área de un objeto de la clase Circle sin valor de retorno como se muestra a continuación.

```
void Circle :: print () const {  
    cout << "Radio: " << radius << endl;  
    cout << "Perimetro: " << 2 * radius * 3.14 << endl;  
    cout << "Area: " << radius * radius * 3.14 << endl;  
}
```

Función miembro mutador

- Son funciones que puedan cambiar el estado de sus objetos anfitriones. No debe tener el calificador const. Los constructores y destructores pueden considerarse funciones miembros mutadoras (inicializan o limpian objetos)
- Por ejemplo, si creamos una clase que representa una cuenta bancaria, el miembro de datos que representa el saldo cambia con el tiempo (con cada depósito y retiro).

```
//no es necesario const para un mutador  
void setRadius (double rds);
```

```
void Circle :: input(){// puede no tener parámetros  
    cout << "Ingrese el radio del objeto Circle: ";  
    cin >> radius;  
}
```

Invariantes de clase

- Es una o más condiciones que debemos imponer a algunos o todos los miembros de datos de instancia de una clase y que debemos hacer cumplir a través de funciones miembro de instancia.
- Ejemplo: en nuestra clase Circle, el radio debe ser un valor positivo. Aplicamos la invariante de una clase a través de funciones miembro de datos que crean objetos (constructores de parámetros) o funciones miembros mutadoras que cambian el valor de un miembro de datos.
- El [Ejemplo 4](#) muestra cómo cambiar el constructor de parámetros de una clase Rectangle para garantizar la invariante de la clase.

Miembros estáticos

- Es un miembro de datos que pertenece a todas las instancias; también pertenece a la clase misma y sus declaraciones deben incluirse en la definición de la clase. Los miembros estáticos usan la palabra clave static.
- Declaración:

```
class Rectangle{  
    private:  
        static int count; // miembro de datos estático  
    public:  
}
```

Miembros estáticos

- Un miembro de datos de instancia normalmente se inicializa en un constructor, pero un miembro de datos estáticos no pertenece a ninguna instancia, lo que significa que no se puede inicializar en un constructor.
- Un miembro de datos estáticos debe inicializarse después de la definición de clase. Esto significa que debe inicializarse en un área global del programa. Debemos demostrar que pertenece a la clase agregando el nombre de la clase y el operador de alcance de la clase (::) a la definición, pero no se debe agregar el calificador estático.
- Inicialización:

```
int Rectangle :: count = 0;
```

Miembros estáticos

- Una función miembro estática puede acceder al miembro de datos estáticos a través de un objeto y también a través del nombre de la clase cuando no existe ningún objeto. Una función miembro estática no tiene ningún objeto host porque no está asociada con ninguna instancia.
- Declaración:

```
//se declara dentro de la clase, usa la palabra static,  
class Rectangle {  
    private:  
        static int count; // miembro de datos estático  
    public:  
        static int getCount(); // función miembro estático  
}
```

Miembros estáticos

- Definición:

//debe definirse fuera de la clase

//== a la def de una función miembro de instancia

//No usamos static. Consultar la declaración para saber el tipo

```
int Rectangle :: getCount() {  
    return count;  
}
```

- Llamada

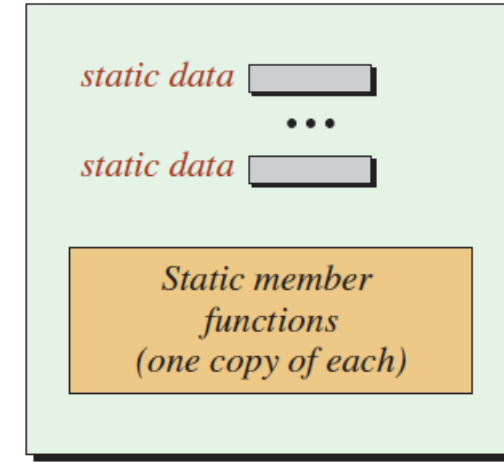
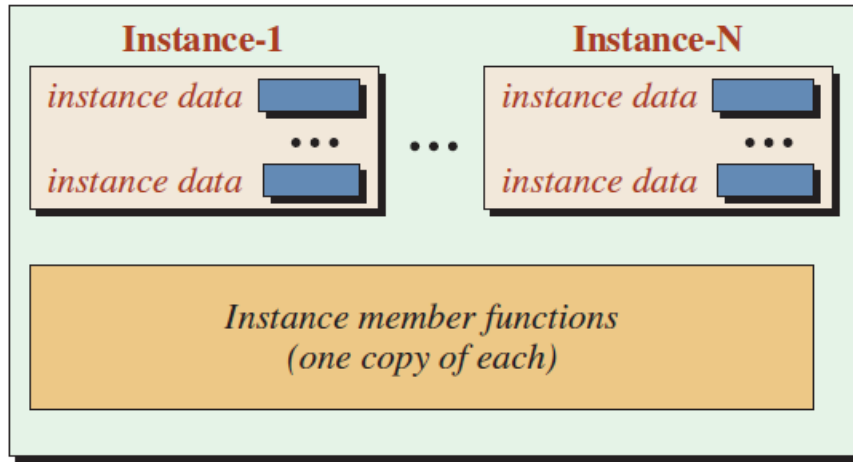
rect.getCount (); // a través de una instancia

Rectangle :: getCount(); // a través de la clase

Miembros estáticos

- **Advertencia:** No podemos usar una función miembro estática para acceder a un miembro de datos de instancia porque una función miembro estática **no tiene el puntero this oculto**, que define la instancia a la que se debe hacer referencia.
- Por otro lado, se puede utilizar una función miembro de instancia para acceder a miembros de datos estáticos (no se utiliza el puntero this),
- Una buena práctica es utilizar funciones de miembros de instancia para acceder a miembros de datos de instancia y funciones de miembros estáticos para acceder a miembros de datos estáticos.

Miembros estáticos



- El Ejemplo 5, muestra un programa con un miembro de datos estáticos y la función miembro estática correspondiente usando la clase Rectangle. El programa también muestra cómo podemos contar las instancias.

Ejercicio: Cuenta bancaria

- Diseñar una clase que representa una cuenta bancaria. Con dos miembros de datos de instancia: el número de cuenta y el saldo. Si bien el usuario de la clase puede inicializar el saldo durante la creación de una instancia, el usuario no puede ingresar el número de cuenta porque debe ser único. Para evitar la duplicación del número de cuenta, utilizar un miembro de datos estáticos, llamado base, que se inicializa a 0 y se incrementa con la apertura de cada nueva cuenta. Agregar 100000 a este miembro de datos estáticos para que sea un número grande.

Ejercicio: Cuenta bancaria

- Debe permitir que la cuenta se cierre cuando finalice el programa (al llamar al destructor, la cuenta se cierra y el saldo restante se envía al banco cliente).
- Debido a la inicialización del número de cuenta debe definir un constructor de parámetros.
- Debe evitar que el sistema cree un constructor copia pues no podemos tener dos cuentas con el mismo número de cuenta (en C++11 declare un constructor de copia y establecerle la palabra clave delete)
- Debe garantizar que no se abra ninguna cuenta con saldo negativo